

# IA 642 Defensive Security

## Week 06, Module 01 — Lab 6.1

### Breaking and Fixing a Serverless API on AWS

#### Cloud & Application/API Security (Free Tier)

---

|                      |                                                                                                                                                                   |               |               |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------------|
| <b>Group members</b> | Unais Ali                                                                                                                                                         | <b>EID</b>    | E02805019     |
|                      | Hafiz Usama                                                                                                                                                       |               | E02574092     |
| <b>Course</b>        | IA 642 Defensive Security                                                                                                                                         | <b>Lab</b>    | 6.1 (Week 06) |
| <b>Date</b>          | October 7, 2026                                                                                                                                                   | <b>Region</b> | us-east-1     |
| <b>AWS account</b>   | 626672433625                                                                                                                                                      | <b>Prefix</b> | ia642-lab     |
| <b>GitHub</b>        | <a href="https://github.com/unaisshazan/ia642-week06-lab61-serverless-api-security">https://github.com/unaisshazan/ia642-week06-lab61-serverless-api-security</a> |               |               |
| <b>Live frontend</b> | <a href="http://ia642-lab-frontend-626672433625.s3-website-us-east-1.amazonaws.com">http://ia642-lab-frontend-626672433625.s3-website-us-east-1.amazonaws.com</a> |               |               |

---

## Lab overview

Meridian Health Network is moving patient-facing account/billing APIs to a serverless tier. This lab deploys a deliberately flawed stack on AWS Free Tier (Cognito + API Gateway HTTP API + Lambda + DynamoDB), proves OWASP API1:2023 Broken Object Level Authorization (BOLA) and API3:2023 Broken Object Property Level Authorization (excessive data exposure), hardens the handler to ASVS v5.0.0 requirements 8.2.2, 8.2.3, and 3.4.x, writes backlog-ready security requirements, and tears every resource down the same day.

## Environment and tooling

- **Identity:** AWS CLI configured for account 626672433625 (root sandbox); verified with `aws sts get-caller-identity` before create/delete.
- **Region / prefix:** us-east-1, resource prefix ia642-lab.
- **Runtime:** Python 3.12 Lambda; Cognito USER\_PASSWORD\_AUTH users alice/bob; DynamoDB on-demand table ia642-lab-accounts (accounts 1001/1002).
- **Automation:** Windows runner run\_lab61.ps1 (JSON via file:// paths); evidence under evidence/.
- **Public repo:** code, evidence captures, frontend, and report PDF (no AWS access keys): <https://github.com/unaisshazan/ia642-week06-lab61-serverless-api-security>
- **Live frontend demo:** Cognito login + GET /accounts/{id} against the Free Tier API: <http://ia642-lab-frontend-626672433625.s3-website-us-east-1.amazonaws.com>
- **Safety / cost:** Free Tier / always-free when torn down same day; no AWS secrets committed.

## Part A — Deploy the vulnerable API

Created an IAM execution role trusted only by `lambda.amazonaws.com`, seeded DynamoDB with alice/bob ownership rows, deployed intentionally vulnerable `vuln_app.handler` (returns any item to any authenticated caller), Cognito user pool/client, and API Gateway route `GET /accounts/{id}` with JWT authorizer. Authentication is enforced at the edge; authorization is not. **Evidence #1**

### — IAM role ARN (create-role)

```
arn:aws:iam::626672433625:role/ia642-lab-role
```

### Supporting deploy artifacts

```
{
  "UserId": "626672433625",
  "Account": "626672433625",
  "Arn": "arn:aws:iam::626672433625:root"
}
Count=2
pool=us-east-1_e7jjz4igA
client=65re926cikt530ktekg4g80ngj
lambda=arn:aws:lambda:us-east-1:626672433625:function:ia642-lab-api
```

### Evidence #2 — API endpoint URL

```
https://r1e1dtd1q6.execute-api.us-east-1.amazonaws.com
```

## Part B — Attack it (BOLA)

No token yields HTTP 401 (JWT authorizer works). Alice's valid `IdToken` reads her own account 1001 successfully. The same token reading Bob's account 1002 also succeeds and returns Bob's balance and `card_last4`. That is OWASP **API1:2023 BOLA**: authentication succeeded; object-level authorization never ran. **Evidence #3 — 401 check, happy path, and BOLA leak**

```
no token -> HTTP 401
ALICE_TOKEN_PREFIX=eyJraWQiOiJZOF1WQ1JzODJsdzZiN3IxS1A5akxJ...
OWN_ACCOUNT={"caller": "alice", "account": {"card_last4": "4242", "balance": "4200", "owner": "alice", "account_id": "1001"}}
BOLA_LEAK={"caller": "alice", "account": {"owner": "bob", "card_last4": "7733", "balance": "98750", "account_id": "1002"}}
```

### Critical thinking #1 — Authentication vs authorization

**Authentication** answers “who are you?” The Cognito JWT authorizer verified Alice's signature, issuer, audience, and `token_use` before Lambda executed. **Authorization** answers “what may you access?” The vulnerable handler never compared `item["owner"]` to `cognito:username`, so any authenticated principal could request any `account_id`. Stronger authentication (longer passwords, MFA, shorter expiry) would still issue a valid Alice token; she would still request 1002 and still receive Bob's billing data. Only server-side object-level authorization closes API1/BOLA. POST/DELETE return 404 because only GET (plus HEAD for header probes) is routed—unused methods are not silently accepted (ASVS v5.0.0-4.1.4). A corrupted JWT signature returns 401 (ASVS v5.0.0-9.1.1). Decoded claims show `token_use=id`, correct aud, Cognito iss, and `cognito:username=alice` (ASVS v5.0.0-9.2.2 / 9.2.3). **Evidence #4 — HTTP methods, tampered token, decoded**

## claims

```
POST -> HTTP 404
DELETE -> HTTP 404
tampered token -> HTTP 401
CLAIMS={"origin_jti":"e594a75d-dde4-4428-850c-21796d291143","sub":"a4a824b8-1071-701e-b2c1-f35ca731f734","aud":"65re926cikt530ktekg4g80ngj","event_id":"80efbac7-cf0a-43ed-bbb2-ea665636dd9a","token_use":"id","auth_time":1791389921,"iss":"https://cognito-idp.us-east-1.amazonaws.com/us-east-1_e7jjz4igA","cognito:username":"alice","exp":1791393521,"iat":1791389921,"jti":"76f62147-259a-4dce-ab9a-3ad44040d62e"}
```

## Map findings to ASVS 5.0

| Observation                 | ASVS 5.0                     | Requirement (paraphrased)                                      | L |
|-----------------------------|------------------------------|----------------------------------------------------------------|---|
| 3.1 no-token → 401          | v5.0.0-8.2.1 / JWT authz     | Unauthenticated callers blocked                                | 1 |
| 3.3 Alice read Bob (BOLA)   | v5.0.0-8.2.2                 | Data-specific access only with explicit permission (IDOR/BOLA) | 1 |
| Enforcement server-side     | v5.0.0-8.3.1                 | Authz at trusted service layer, not client                     | 1 |
| POST/DELETE → 404           | v5.0.0-4.1.4                 | Only explicitly supported HTTP methods                         | 3 |
| Tampered token → 401        | v5.0.0-9.1.1                 | Validate signature/MAC before trust                            | 1 |
| aud / token_use checked     | v5.0.0-9.2.2 / 9.2.3         | Correct token type and audience                                | 2 |
| Full item returned (Part D) | v5.0.0-8.2.3                 | Expose only authorized fields                                  | 2 |
| HSTS / nosniff / CSP        | v5.0.0-3.4.1 / 3.4.4 / 3.4.3 | Browser security headers                                       | 1 |

## Part C — Fix BOLA and add security headers

Object-level check: if `item.get("owner") != caller`, return HTTP 403. Alice still reads 1001 (200); Bob's 1002 is denied (403). This is ASVS v5.0.0-8.2.2. **Code diff — BOLA fix (vuln\_app.py ightarrow fixed\_app\_bola\_only.py)**

```
diff --git a/vuln_app.py b/fixed_app_bola_only.py
index 73eebfa..056dd8d 100644
--- a/vuln_app.py
+++ b/fixed_app_bola_only.py
@@ -9,8 +9,12 @@ def handler(event, context):
    item = table.get_item(Key={"account_id": acct_id}).get("Item")
    if not item:
        return {"statusCode": 404, "body": json.dumps({"error": "not found"})}
-    # VULNERABILITY (OWASP API1:2023 BOLA): returns the record to ANY
-    # authenticated caller -- no check that item["owner"] == caller.
+    # FIX (ASVS v5.0.0-8.2.2): object-level authorization -- return the record
+    # only if the authenticated caller actually owns it.
+    if item.get("owner") != caller:
+        return {"statusCode": 403,
+                "headers": {"Content-Type": "application/json"},
+                "body": json.dumps({"error": "forbidden", "caller": caller})}
```

```
return {"statusCode": 200,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps({"caller": caller, "account": item}, default=str)}
```

## Evidence #5 — Post-fix retest (200 / 403)

```
OWN={"caller": "alice", "account": {"card_last4": "4242", "balance": "4200", "owner": "alice", "
    account_id": "1001"}} <- HTTP 200
BOB={"error": "forbidden", "caller": "alice"} <- HTTP 403
```

Responses also carry ASVS V3 headers: Strict-Transport-Security (3.4.1), X-Content-Type-Options: nosniff (3.4.4), restrictive Content-Security-Policy (3.4.3/3.4.6), and Cache-Control: no-store. **Evidence #6 — Response headers (curl -sI)**

```
HTTP/1.1 200 OK
Date: Wed, 07 Oct 2026 16:19:29 GMT
Content-Type: application/json
Content-Length: 95
Connection: keep-alive
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
content-security-policy: default-src 'none'; frame-ancestors 'none'; base-uri 'none'
cache-control: no-store
Apigw-Requestid: E4aa1hycIAMEJyg=
```

## Critical thinking #2 — ASVS levels vs severity

ASVS Level 1/2/3 is an **assurance target** for how comprehensively an application should be verified, not a ranking of how damaging a failed control is. HSTS (3.4.1) and object-level authorization (8.2.2) can both appear at L1 because both are baseline expectations for many apps, yet their business impact differs. Organizations choose L1/L2/L3 from data sensitivity, threat model, and regulation. Meridian handles PHI under HIPAA: a billing API that can exfiltrate another patient's account is a reportable privacy incident. Meridian should target at least L2 for any PHI-touching API (and L3 for high-assurance clinical systems), treat 8.2.2 and 8.2.3 as release blockers, and still ship L1 browser headers because the same origin feeds a web portal—without pretending headers equal object authorization in risk.

## Security requirements table

| ID   | Security requirement (testable)                                                                                                                      | ASVS                         | Verification                                                                 |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|------------------------------------------------------------------------------|
| SR-1 | Every endpoint returning a user-owned object MUST verify the authenticated caller owns that object before returning it.                              | v5.0.0-8.2.2                 | User A requests user B's object → 403; owner → 200.                          |
| SR-2 | All API responses rendered by a browser MUST include HSTS, nosniff, and a restrictive CSP.                                                           | v5.0.0-3.4.1 / 3.4.4 / 3.4.3 | <code>curl -I</code> asserts the three headers on every 2xx.                 |
| SR-3 | Account-read routes MUST accept only explicitly configured HTTP methods (GET; HEAD only if required). POST/PUT/PATCH/DELETE MUST not be auto-mapped. | v5.0.0-4.1.4                 | Probe with POST/DELETE using a valid token; expect 404/405.                  |
| SR-4 | API Gateway MUST reject JWTs with invalid signatures, wrong aud, wrong iss, or wrong token_use before Lambda executes.                               | v5.0.0-9.1.1 / 9.2.2 / 9.2.3 | Corrupted signature → 401; decode valid claims and assert aud/iss/token_use. |

## Tear down

Deleted API Gateway API, Lambda, DynamoDB table, Cognito user pool, IAM inline policy and role. Confirmation queries for prefix `ia642-lab` returned empty lists. **Evidence #7 — Empty teardown confirmation**

```
SKIPPED - live demo left running
ENDPOINT=https://r1e1dtd1q6.execute-api.us-east-1.amazonaws.com
POOL_ID=us-east-1_e7jjz4igA
CLIENT_ID=65re926cikt530ktekg4g80ngj
API_ID=r1e1dtd1q6
```

## Part D — Graded challenge (excessive data exposure)

### 1. Vulnerability class

Returning the whole DynamoDB item is OWASP API Top 10 (2023) **API3:2023 Broken Object Property Level Authorization** (excessive data exposure). Corresponding ASVS field-level requirement: **v5.0.0-8.2.3**—return only properties the consumer is authorized to see.

### 2. Exploit

After the BOLA ownership fix, sensitive attributes were added to account 1001 via `dynamodb update-item (internal_note=VIP-PHI-REVIEW, fraud_score=0.91)`. Alice's authorized GET still leaked them because the handler serialized `item` wholesale: **Part D exploit — authorized owner, still leaking fields**

```
{"caller": "alice", "account": {"account_id": "1001", "balance": "4200", "card_last4": "4242", "fraud_score": "0.91", "internal_note": "VIP-PHI-REVIEW", "owner": "alice"}}
```

### 3. Allow-list fix

Response built from `ALLOWED_FIELDS = ("account_id", "balance", "card_last4")` only. After redeploy, `internal_note` and `fraud_score` disappear. An allow-list is safer than a deny-list because new sensitive columns added later are excluded by default; a blocklist must be updated every time the schema grows and will miss fields nobody remembered to ban. **Part D fixed output — allow-list**

```
{"caller": "alice", "account": {"account_id": "1001", "balance": "4200", "card_last4": "4242"}}
```

#### Code diff — allow-list + security headers

```
diff --git a/fixed_app_bola_only.py b/fixed_app.py
index 056dd8d..f9e5658 100644
--- a/fixed_app_bola_only.py
+++ b/fixed_app.py
@@ -1,20 +1,22 @@
 import json, boto3
 ddb = boto3.resource("dynamodb")
 table = ddb.Table("ia642-lab-accounts")
-
+ALLOWED_FIELDS = ("account_id", "balance", "card_last4")
+SECURE_HEADERS = {
+    "Content-Type": "application/json",
+    "Strict-Transport-Security": "max-age=31536000; includeSubDomains",
+    "X-Content-Type-Options": "nosniff",
+    "Content-Security-Policy": "default-src 'none'; frame-ancestors 'none'; base-uri 'none'",
+    "Cache-Control": "no-store",
+}
+
 def handler(event, context):
     claims = event.get("requestContext", {}).get("authorizer", {}).get("jwt", {}).get("claims", {})
     caller = claims.get("cognito:username", "anonymous")
     acct_id = event.get("pathParameters", {}).get("id")
     item = table.get_item(Key={"account_id": acct_id}).get("Item")
     if not item:
-        return {"statusCode": 404, "body": json.dumps({"error": "not found"})}
-        # FIX (ASVS v5.0.0-8.2.2): object-level authorization -- return the record
-        # only if the authenticated caller actually owns it.
+        return {"statusCode": 404, "headers": SECURE_HEADERS, "body": json.dumps({"error": "not found"})}
     if item.get("owner") != caller:
-        return {"statusCode": 403,
-            "headers": {"Content-Type": "application/json"},
-            "body": json.dumps({"error": "forbidden", "caller": caller})}
-        return {"statusCode": 200,
-            "headers": {"Content-Type": "application/json"},
-            "body": json.dumps({"caller": caller, "account": item}, default=str)}
+        return {"statusCode": 403, "headers": SECURE_HEADERS, "body": json.dumps({"error": "forbidden", "caller": caller})}
+        safe = {k: item[k] for k in ALLOWED_FIELDS if k in item}
+        return {"statusCode": 200, "headers": SECURE_HEADERS, "body": json.dumps({"caller": caller, "account": safe}, default=str)}
```

## 4. Least privilege

Section 2.2 granted the Lambda role broad DynamoDB actions on `Resource "*"`. To satisfy least privilege: (a) grant only `dynamodb:GetItem` for a read-only API; (b) set `Resource` to the specific table ARN `arn:aws:dynamodb:us-east-1:626672433625:table/ia642-lab-accounts` (plus index ARNs if needed); (c) omit `PutItem/UpdateItem/Scan/Query` unless required. That control belongs to ASVS V15 **Secure Coding & Architecture** (least-privilege / secure cloud IAM configuration), with related trust-boundary themes in V1—not to browser headers.

## Final hardened handler

### fixed\_app\_final.py (BOLA + allow-list + ASVS headers)

```
import json, boto3
ddb = boto3.resource("dynamodb")
table = ddb.Table("ia642-lab-accounts")
ALLOWED_FIELDS = ("account_id", "balance", "card_last4")
SECURE_HEADERS = {
    "Content-Type": "application/json",
    "Strict-Transport-Security": "max-age=31536000; includeSubDomains",
    "X-Content-Type-Options": "nosniff",
    "Content-Security-Policy": "default-src 'none'; frame-ancestors 'none'; base-uri 'none'",
    "Cache-Control": "no-store",
}

def handler(event, context):
    claims = event.get("requestContext", {}).get("authorizer", {}).get("jwt", {}).get("claims", {})
    caller = claims.get("cognito:username", "anonymous")
    acct_id = event.get("pathParameters", {}).get("id")
    item = table.get_item(Key={"account_id": acct_id}).get("Item")
    if not item:
        return {"statusCode": 404, "headers": SECURE_HEADERS, "body": json.dumps({"error": "not found"})}
    if item.get("owner") != caller:
        return {"statusCode": 403, "headers": SECURE_HEADERS, "body": json.dumps({"error": "forbidden", "caller": caller})}
    safe = {k: item[k] for k in ALLOWED_FIELDS if k in item}
    return {"statusCode": 200, "headers": SECURE_HEADERS, "body": json.dumps({"caller": caller, "account": safe, "default": str})}
```

**Conclusion.** Authentication alone did not protect Meridian's accounts. Object-level authorization (8.2.2) stopped cross-patient reads; field allow-listing (8.2.3 / API3) stopped schema leakage; method restriction, JWT validation, and security headers completed the ASVS map. Written SRs turn these fixes into backlog-ready acceptance tests. Stack torn down same day — Free Tier cost remains \$0.